

Solidity



Solidity:

- Solidity è un linguaggio di programmazione orientato agli oggetti, sviluppato per scrivere smart contract sulla blockchain Ethereum.
- Viene paragonato a JavaScript, ma assomiglia di più a C.
- È un linguaggio **fortemente tipizzato** con funzionalità minimali che enfatizza la limitazione dell'uso di memoria e CPU.
- Supporta l'ereditarietà, le librerie e i tipi di dato complessi.

<https://docs.soliditylang.org/en/v0.8.23/>

Ambiente di sviluppo

- Solidity può essere utilizzato con vari ambienti di sviluppo integrato (IDE) come Remix, Truffle Suite, e Visual Studio Code.
- Gli strumenti di testing e debugging sono essenziali per garantire la sicurezza e l'affidabilità degli smart contract.



Remix

Remix:

- Ambiente di sviluppo integrato (IDE) su browser per lo sviluppo di smart contract in Solidity.
- Integra una blockchain virtuale (Ethereum JavaScript VM) per testare e distribuire smart contract direttamente dall'IDE.
- Strumenti di logging per tracciare e analizzare le transazioni.

<https://remix.ethereum.org/>





EthFiddle:

- Ethereum Fiddle è uno strumento online per scrivere, condividere e testare frammenti di codice Solidity.
- È simile a JSFiddle, ma specificamente progettato per Solidity.
- Offre la possibilità di eseguire e testare codice Solidity all'interno del browser, senza la necessità di un ambiente di sviluppo locale.
- Utile per l'apprendimento, la sperimentazione e la dimostrazione di specifiche funzionalità di Solidity.

<https://ethfiddle.com/>



Primo contratto

```
pragma solidity ^0.8.0; //Specifica la versione del compilatore
contract SimpleCounter //Dichiarazione del contratto, simile alla definizione di una classe in altri linguaggi
{
    uint private _count; //Variabile memorizzata sulla blockchain
    constructor() //Viene chiamato quando il contratto viene deployato
    {
        _count = 10;
    }
    function increment() public
    {
        _count += 1;
    }
    function getCount() public view returns (uint)
    {
        return _count;
    }
}
```

Primo contratto

Il contratto SimpleCounter è un semplice esempio di come mantenere e manipolare lo stato in un contratto Solidity.

- Una variabile privata tiene traccia del conteggio.
- Il costruttore imposta il valore iniziale di count a 10 quando il contratto viene distribuito.
- La funzione pubblica `increment()` incrementa il valore di `_count` di 1 ogni volta che viene chiamata.
- La funzione pubblica di sola lettura (view) `getCount()` restituisce il valore attuale di `_count`.

Testing

Test che verificano le funzionalità di base del contratto e assicurano che il comportamento sia quello atteso.

- Deploy del contratto: dopo aver inserito il codice del contratto in Remix, compilarlo e fare il deploy. Verificare che non ci siano errori e che il contratto sia stato distribuito con successo.
- Verifica del valore iniziale di `_count`: chiamare la funzione `getCount()` e verificare che restituisca il valore iniziale di 10.
- Test dell'incremento: invocare la funzione `increment()` e chiamare di nuovo `getCount()` per assicurarsi che il contatore sia aumentato di 1.
- Ripetere il test precedente per verificare che il contatore continui ad aumentare correttamente ogni volta.



Deployment:

- Dopo aver compilato il codice sorgente in bytecode, utilizzando strumenti come *Truffle* o script di deployment, il bytecode viene **distribuito** sulla blockchain.
- Il deployment avviene inviando una transazione all'**indirizzo nullo** (0x0...) con il bytecode come dati.
- Per il deployment è necessario **pagare** una quantità di gas, che è il costo associato all'esecuzione delle operazioni sulla rete Ethereum.
- Al termine del deployment, al contratto verrà assegnato un **indirizzo unico** sulla blockchain, che deve essere utilizzato per interagire con esso.



Application Binary Interface (ABI):

- L'Application Binary Interface (**ABI**) è l'**interfaccia** tra due programmi binari (equivalente in binario o bytecode di un'API): uno smart contract e il codice che lo chiama, che può essere un altro contratto o un'applicazione client.
- Un ABI contiene la **firma** di ciascuna funzione pubblica ed esterna dello smart contract, includendo il nome della funzione, i tipi di parametri in ingresso e i tipi di ritorno. Esclude le funzioni private e interne.
- L'ABI viene generato automaticamente al momento della compilazione del codice e può essere estratto utilizzando strumenti come solc (Solidity compiler) o ambienti di sviluppo integrati.
- Le applicazioni client utilizzano l'ABI per **costruire le transazioni** che invocano le funzioni dello smart contract, e per interpretare gli eventi.

```
{
    "inputs": [],
    "name": "increment",
    "outputs": [],
    "stateMutability": "nonpayable",
    "type": "function"
},
{
    "inputs": [],
    "stateMutability": "nonpayable",
    "type": "constructor"
},
{
    "inputs": [],
    "name": "getCount",
    "outputs": [
        {
            "internalType": "uint256",
            "name": "",
            "type": "uint256"
        }
    ],
    "stateMutability": "view",
    "type": "function"
}
]
```

[illegible]

Bytecode

- **generatedSources**: si riferisce a porzioni di **codice supplementare** create automaticamente dal compilatore (wrapper, interazione con altri contratti, codice aggiuntivo per la conformità agli standard o per l'ottimizzazione).
- **linkReferences**: usato se il contratto dipende da **librerie esterne**. Durante la compilazione queste dipendenze non sono ancora risolte, e questo campo indica dove queste dipendenze devono essere collegate.
- **object: bytecode** effettivo del contratto che rappresenta il codice che viene eseguito sulla Ethereum Virtual Machine e distribuito sulla blockchain.
- **opcodes**: rappresentazione **leggibile** degli opcode del bytecode. Ogni opcode esegue un'operazione specifica all'interno dell'EVM.
- **sourceMap**: guida che collega il bytecode agli specifici **pezzi di codice sorgente** Solidity da cui è derivato. Può essere utile quando si utilizzano strumenti di debugging che possono mostrare quale parte del codice sorgente corrisponde a quale parte del bytecode in caso di errore.

Secondo contratto

```
pragma solidity ^0.8.0; //Specifica la versione del compilatore
contract SimplePonzi
{
    address payable _currentInvestor;
    uint256 public _currentInvestment = 0; //non occorre inizializzare le variabili a zero, è uno spreco di gas!
    receive() external payable public
    {
        uint256 minimumInvestment = _currentInvestment * 11/10; //il nuovo investimento deve essere > 10% di quello corrente
        require(msg.value > minimumInvestment);
        address payable previousInvestor = _currentInvestor;
        _currentInvestor = payable(msg.sender);
        _currentInvestment = msg.value;
        previousInvestor.transfer(msg.value);
    }
}
```

Simple Ponzi

Il contratto SimplePonzi rappresenta un semplice esempio di **schema Ponzi**, dove ogni nuovo partecipante paga il precedente.

Tiene traccia dell'indirizzo dell'ultimo investitore e dell'ammontare del suo investimento.

La Funzione `Receive` impone che l'importo da inviare per partecipare sia almeno il **10%** più grande di quello corrente.

Se la condizione è soddisfatta:

- L'indirizzo dell'investitore corrente viene aggiornato con l'indirizzo del mittente della transazione,
- l'investimento corrente viene aggiornato all'ammontare ricevuto,
- l'investitore precedente riceve l'importo inviato al contratto.

Testing

Test che assicurano che il contratto gestisca gli investimenti come previsto e che i pagamenti vengano effettuati correttamente.

- Verificare che `_currentInvestor` sia l'indirizzo zero e `_currentInvestment` sia uguale a zero.
- Fare un investimento inviando Ether al contratto e verificare che `_currentInvestor` e `_currentInvestment` vengano aggiornati correttamente.
- Provare a inviare un valore minore di `_currentInvestment * 1.1` e verificare che la transazione venga respinta.
- Dopo un nuovo investimento fatto con successo, controllare che l'investitore precedente abbia ricevuto i fondi.
- Verificare che i nuovi investitori ricevano i pagamenti e che il requisito dell'incremento del 10% venga rispettato.

Variabili

Variabili di Storage:

- **Costo elevato.**
- Memorizzate **permanentemente** sulla blockchain.
- Dichiarate a **livello di contratto** (al di **fuori** delle funzioni) e non è necessario usare una parola chiave specifica.
- Si può usare il modificatore **storage** all'interno delle funzioni.

Variabili volatili:

- Economiche.
- Non resta traccia nella blockchain (esistenza è **limitata** alla durata della chiamata alla funzione).
- Memory:
 - Dichiarate a livello di **funzione**.
 - Si può usare il modificatore **memory** per tipi di dati **complessi** come array e struct).
- Calldata:
 - Utilizzate principalmente per i parametri delle funzioni **esterne**
 - Immutabili e più **efficienti** in termini di **gas** rispetto a memory.
 - Dichiarate esplicitamente con il modificatore **calldata** nelle funzioni **external**.

Terzo contratto

```
contract MemoryStorageCalldata
{
    uint[5] public numbers = [1, 2, 3, 4, 5];

    function memoryWorking() public view returns (uint[5] memory)
    {
        uint[5] memory A = numbers;
        A[0] = 99;
        return numbers;
    }

    function storageWorking() public returns (uint[5] memory)
    {
        uint[5] storage B = numbers;
        B[0] = 99;
        return numbers;
    }

    function calldataWorking(uint[] calldata _input) external view returns
    (uint[5] memory)
    {
        require(_input.length > 0, "wrong number of parameters");

        uint[5] memory A = numbers;
        A[0] = _input[0];

        return numbers;
    }
}
```



Booleani: `bool`.

Interi:

- `int` / `int256`: intero con segno a 256 bit.
- `uint` / `uint256`: intero senza segno a 256 bit.
- Versioni più piccole `int8`, `uint8`, `int16`, `uint16`, ecc

Indirizzi:

- `address`: contiene un indirizzo Ethereum a 20 byte.
- `address payable`: variante di `address` che può ricevere Ether.

Byte e Stringhe:

- `bytes1`, ..., `bytes32`: dati binari di lunghezza fissa.
- `bytes`: dati binari di lunghezza variabile.
- `string`: stringa di caratteri di lunghezza variabile.

Enumerazione: `enum`.



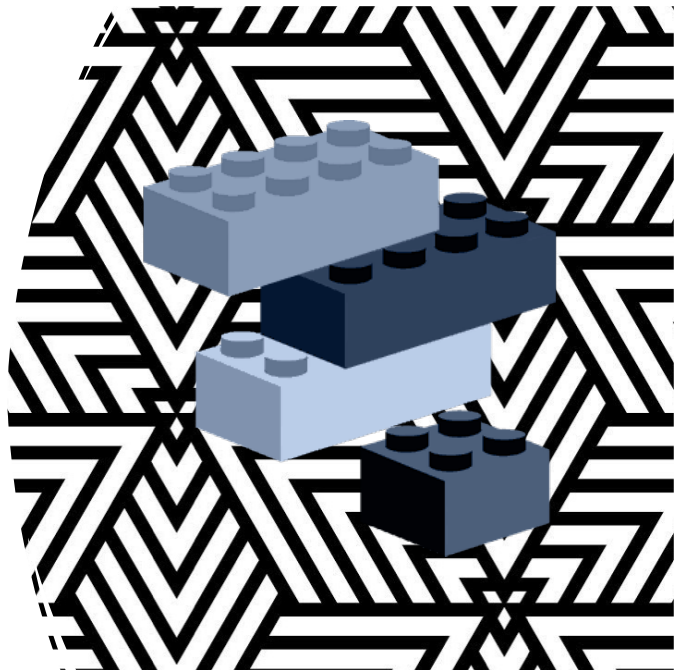
Esistono anche tipi per numeri decimali fissi:

- `fixed`, `ufixed`, `fixedMxN` e `ufixedMxN`, ma non sono supportati ancora.
- Necessità di evitare qualsiasi ambiguità o imprecisione nelle operazioni matematiche.
- I numeri **decimali** sono gestiti utilizzando **interi**, **scalando** i valori per un fattore fisso.
- Esempio: 1.5 Ether può essere rappresentato come 150000000000000000 wei.
- Solidity e l'EVM non sono ottimizzati per calcoli matematici complessi, soprattutto quelli che richiedono precisione decimale elevata.

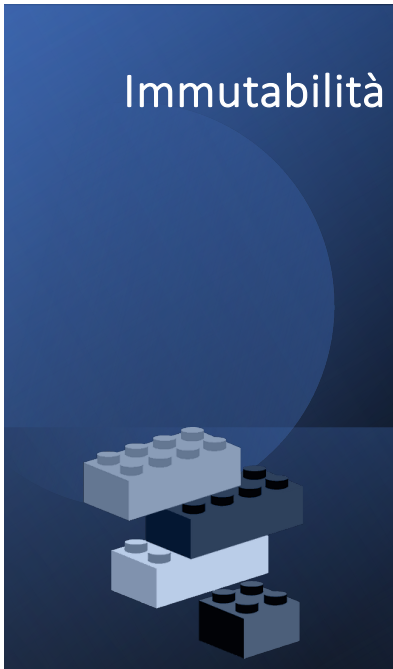
Visibilità

- **Private:** funzioni e variabili private sono accessibili solo all'interno del contratto in cui sono definite.
- **Internal:** funzioni e variabili internal sono accessibili dai contratti derivati.
- **Public:** funzioni e variabili public sono accessibili sia all'interno che all'esterno del contratto. Per le variabili, Solidity crea automaticamente una funzione **getter**.
- **External:** le funzioni external sono accessibili solo dall'esterno e non possono essere chiamate internamente.
 - aiuta a definire un'**interfaccia contrattuale pulita e comprensibile**,
 - i dati vengono passati utilizzando `calldata`, che è un'area di memoria **meno costosa** rispetto alla `memory`, portando a un **risparmio significativo di gas**.
- **View:** le funzioni view promettono di non modificare lo stato del contratto, permettendo **solo la lettura** delle variabili.
 - non effettuano alcuna transazione sulla blockchain. (**Assenza di costi di gas.**)
- **Pure:** le funzioni pure assicurano che **non leggono** lo stato del contratto.
 - utilizzate per eseguire calcoli o restituire valori che dipendono dai parametri forniti.
- **Payable:** Le funzioni payable possono ricevere Ether insieme alla chiamata.

Desing Pattern



Immutabilità



Owner:

- Il pattern *Owner* consiste nel memorizzare l'indirizzo del deployer del contratto, che viene considerato il **proprietario** del contratto.
- Questo indirizzo viene letto dalla proprietà `msg.sender` e impostato all'interno di una variabile di tipo `address` dal metodo costruttore, che viene automaticamente chiamato quando il contratto viene distribuito sulla blockchain.
- Verificando se l'indirizzo dell'utente che invoca un metodo del contratto corrisponde all'indirizzo del proprietario memorizzato, permette di effettuare alcune operazioni (critiche) solo se eseguite dal proprietario.
- Pone le basi per operazioni di **autenticazione** e gestione del contratto.

Owner

```
pragma solidity ^0.8.0; //Specifica la versione del compilatore
contract Ownable //Dichiarazione del contratto, simile alla definizione di una classe in altri linguaggi
{
    address private _owner; //Variabile di tipo address, memorizzerà un indirizzo di un wallet
    uint256 private _state; //Non inizializzo la variabile se devo porla a zero perché è uno spreco di gas
    //Esistono anche interi più piccoli, ma è uno spreco di gas perché vengono comunque convertiti in uint256

    constructor() //Viene chiamato quando il contratto viene deployato
    {
        _owner = msg.sender;
    }
}
```

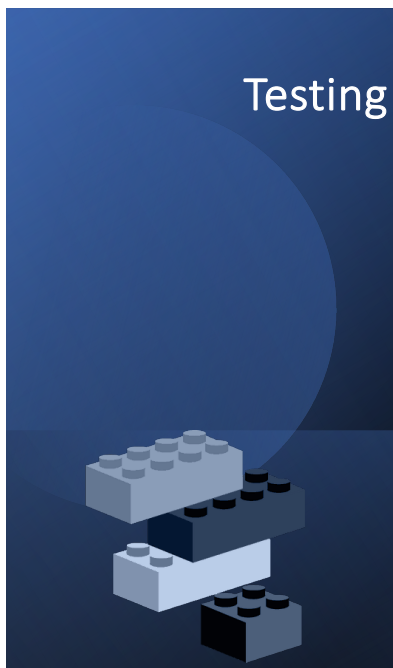
```

function getOwner() public view returns (address)
{
    return _owner;
}

function getState() public view returns (uint256)
{
    return _state;
}

function setState(uint256 _value) public
{
    require(msg.sender == _owner, "Not owner!");
    _state = _value;
}
}

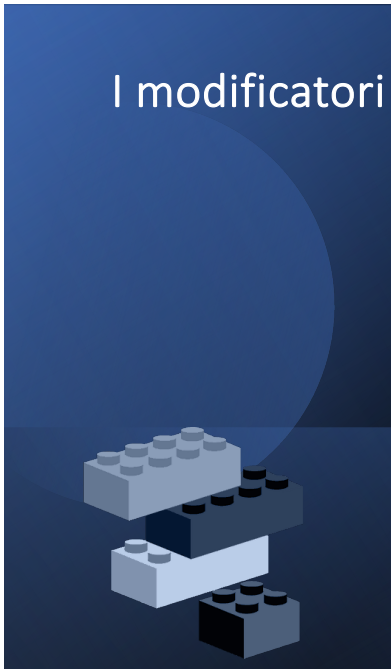
```



Test per il contratto Ownable.

- **Verifica del proprietario:** dopo il deploy, chiamare la funzione `getOwner()` e verificare che l'indirizzo restituito sia uguale all'indirizzo che ha distribuito il contratto.
- **Verifica dello stato iniziale:** chiamare la funzione `getState()` e verificare che il valore iniziale di `_state` sia 0, dato che le variabili in Solidity sono inizializzate a zero per default.
- **Cambio di stato:** chiamare la funzione `setState()` con un valore nuovo usando l'indirizzo del proprietario e verificare che il valore di `_state` sia aggiornato correttamente chiamando di nuovo `getState()`.
- **Controllo del proprietario:** provare a chiamare `setState()` con un indirizzo diverso dal proprietario e verificare che la transazione venga respinta con l'errore "Not owner!".

I modificatori



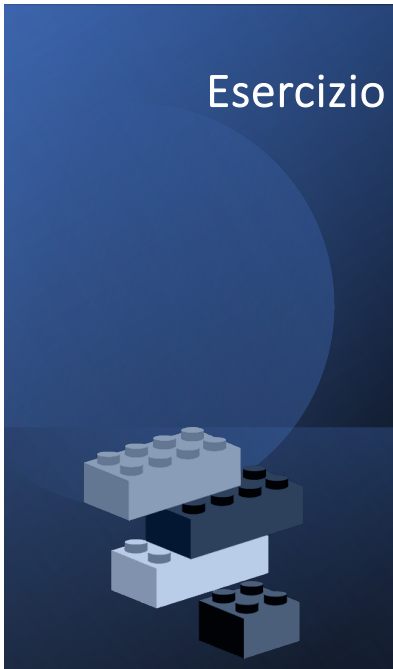
- Un modificatore in Solidity è una funzione speciale che viene utilizzata per modificare il comportamento delle funzioni nello smart contract.
- Viene impiegato per aggiungere un **controllo pre-condizione** prima dell'esecuzione di una funzione.
- Esempio di Modificatore:

```
modifier onlyOwner()  
{  
    require(msg.sender == owner, "Not owner!");  
    _; //Continua l'esecuzione della funzione  
}
```
- Questo modificatore assicura che una funzione possa essere chiamata solo dal proprietario dello smart contract:

Owner con modificatore

```
modifier onlyOwner()  
{  
    require(msg.sender == _owner, "Not owner!");  
    _; //Continua l'esecuzione della funzione  
}  
  
function setState(uint256 _value) public onlyOwner //Viene usato il modificatore che integra il controllo nella funzione  
{  
    _state = _value;  
}
```

Esercizio

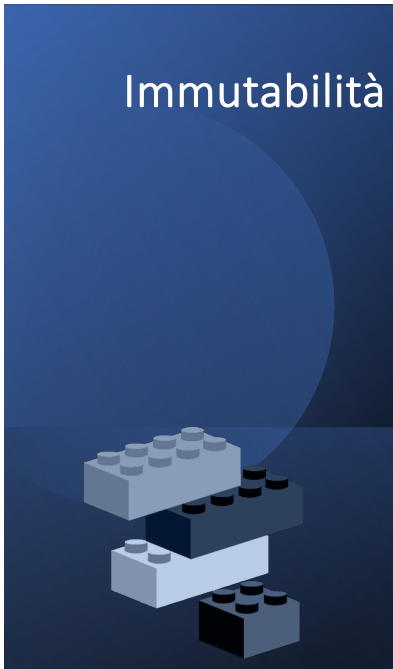


- Aggiungere al contratto Owner.sol la possibilità di cedere la proprietà del contratto ad un altro wallet.

Owner2

```
function changeOwner(address _newOwner) public onlyOwner
{
    _owner = _newOwner;
}
```

Immutabilità



Authorization:

- A differenza del pattern Owner, che conferisce tutti i diritti ad un unico account, il pattern *Authorization* permette di distribuire i **privilegi su diversi livelli**, ad una pluralità di utenti autorizzati.
- Utilizza una mappatura di indirizzi, tipicamente implementata con una hash table (**dizionario**), per memorizzare gli indirizzi dei *super utenti* che possono eseguire **azioni specifiche**, modificare lo stato del contratto o attivare funzioni critiche.
- Questo fornisce un controllo più granulare e una distribuzione più equilibrata delle responsabilità, **riducendo il rischio di abuso** di potere da parte di un singolo proprietario.

Authorization

```
pragma solidity ^0.8.0;
contract Authorization
{
    mapping(address => uint256) public permissions; //Mapping per tracciare i livelli di permesso per ogni indirizzo

    constructor()
    {
        permissions[msg.sender] = 3; // Imposta il livello di permesso massimo per il deployer
    }

    modifier hasPermission(uint256 _level) // Modificatore per controllare il livello di permesso
    {
        require(permissions[msg.sender] >= _level, "Permission not granted!");
        _;
    }

    function getPermission(address _user) public view returns(uint256)
    {
        return permissions[_user];
    }
}
```

```

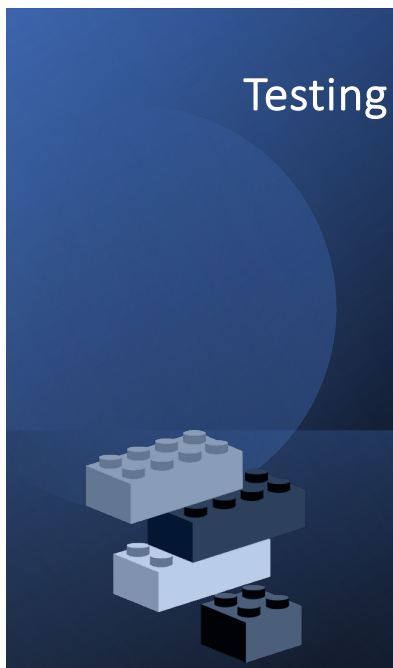
function setPermission(address _user, uint256 _level) public hasPermission(3) //cambia il livello per un indirizzo
{
    permissions[_user] = _level;
}

function functionLevel1() public hasPermission(1) //Richiede un livello di permesso 1 per essere eseguita
{
    // Logica della funzione
}

function functionLevel2() public hasPermission(2) //Richiede un livello di permesso 2 per essere eseguita
{
    // Logica della funzione
}

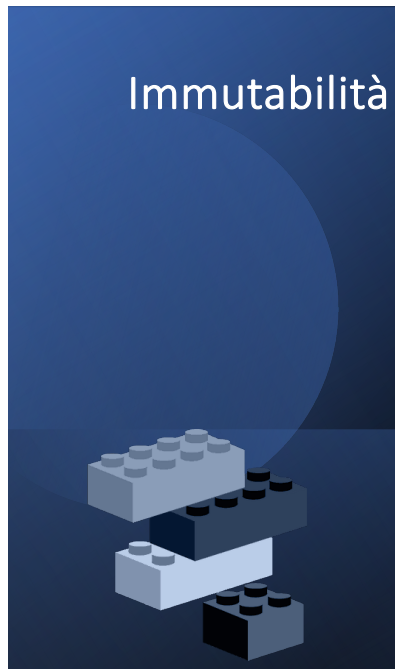
function functionLevel3() public hasPermission(3) //Richiede un livello di permesso 3 per essere eseguita
{
    // Logica della funzione
}
}

```



Test per verificare la logica di autorizzazione del contratto Authorization.

- **Verifica permesso del deployer:** dopo il deploy del contratto, verificare che il livello di permesso per il deployer sia 3 utilizzando la funzione `getPermission()`.
- **Nuovi permessi:** chiamare `setPermission()` per altri indirizzi con livelli di permesso diversi e verificare che siano stati aggiornati correttamente con `getPermission()`.
- **Accesso alle funzioni:** chiamare `functionLevel1()`, `functionLevel2()`, e `functionLevel3()` con indirizzi che hanno il permesso adeguato e verificare che vengano eseguite senza errori.
- **Controllo dei permessi di accesso:** provare a chiamare le funzioni `functionLevel1()`, `functionLevel2()`, `functionLevel3()` e `setPermission()` con indirizzi che non hanno il livello di permesso richiesto e assicurarsi che le transazioni vengano respinte.



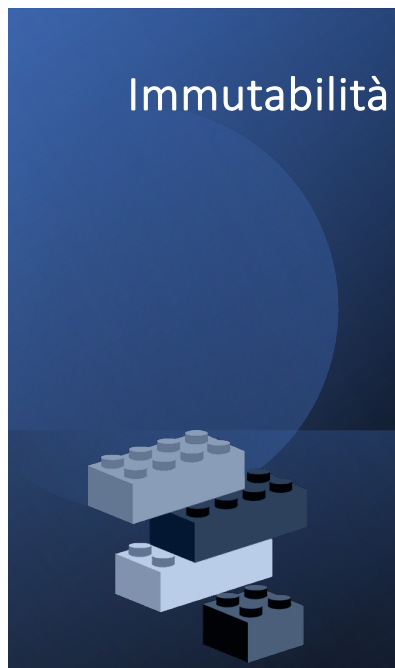
Immutabilità

Contract Composer:

- Il Contract Composer si basa sulla suddivisione di uno smart contract complesso in più **contratti più piccoli e specializzati**.
- Ogni contratto ha una **responsabilità definita e chiara**, rendendo il sistema più **modulare** e gestibile.
- Permette anche una maggiore flessibilità e **riusabilità** dei componenti, che possono essere adattati e riutilizzati in diversi progetti o parti del progetto.

Façade:

- Il Façade Pattern consiste nel creare un contratto che fornisce un'**interfaccia** e gestisce l'interazione con contratti componenti, **nascondendone la complessità**.



Immutabilità

L'esempio implementa un sistema di voto decentralizzato su blockchain con funzionalità di sicurezza e gestione. Façade funge da interfaccia verso VoteRecorder che registra i voti e VoteTracker che gestisce i votanti.

Ownable:

- definisce il proprietario del contratto,
- fornisce il modificatore onlyOwner che restringe l'accesso a certe funzioni.

VoteBaseComposed (eredita da Ownable):

- ha una funzione per impostare l'indirizzo del contratto façade,
- fornisce il modificatore onlyFacade,
- ha una funzione per terminare la votazione.

VoteRecorder (eredita da VoteBaseComposed):

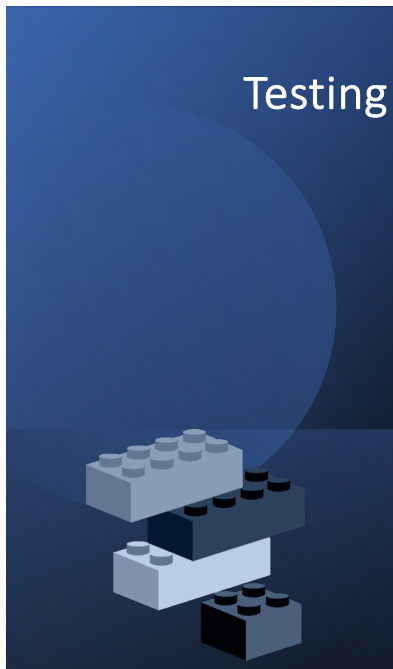
- fornisce un modificatore per controllare se le votazioni sono chiuse,
- include una funzione che restituisce il numero di soluzioni votabili,
- include una funzione per aggiungere soluzioni votabili,
- fornisce un modificatore per controllare se le soluzioni votate esistono,
- registra i voti per le soluzioni,
- permette di recuperare i risultati del voto.

VoteTracker (eredita da VoteBaseComposed):

- fornisce un modificatore per controllare se le votazioni sono aperte,
- ha una funzione per registrare l'indirizzo di chi ha votato,
- ha una funzione per ottenere il numero totale di votanti.

VotingFacade (eredita da Ownable):

- interfaccia pubblica per VoteTracker e VoteRecorder.



Testing

Ownable:

- verificare che il proprietario sia impostato correttamente al deploy del contratto.

VotBaseComposed:

- controllare che `_facadeAddress` possa essere impostato correttamente.
- verificare che solo il Facade possa chiamare `closeVoting()`.

VoteRecorder:

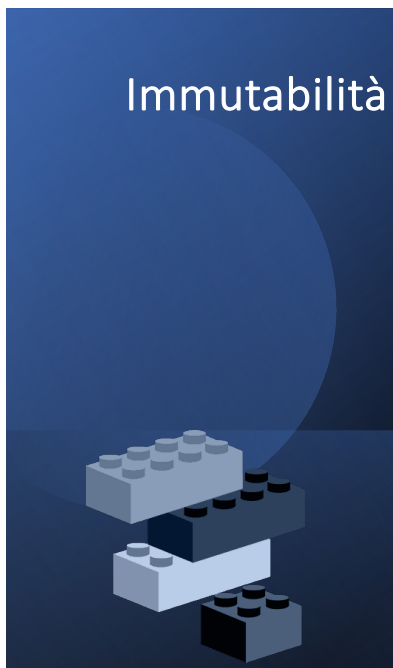
- testare l'aggiunta di soluzioni e verificarne il conteggio.
- votare per una soluzione e assicurarsi che il conteggio dei voti venga incrementato.
- controllare che il voto non sia possibile se il voto è chiuso o se la soluzione non rientra nell'intervallo corretto.

VoteTracker:

- testare che un indirizzo possa registrare il voto e che non possa votare più volte.
- verificare che il conteggio totale dei votanti venga aggiornato dopo ogni voto valido.

VotingFacade:

- assicurarsi che tutte le funzioni reindirizzino correttamente le chiamate a `VoteTracker` e `VoteRecorder` e che funzionino come previsto.
- verificare che la chiusura del svoto possa essere eseguita solo dal proprietario.



Immutabilità

Migration:

- Il pattern *Migration*, anche noto come *Proxy*, permette di affrontare la limitazione dell'**immutabilità** degli smart contract, offrendo una maggiore flessibilità nella gestione e nell'aggiornamento dei contratti.
- Viene utilizzato un contratto proxy, che memorizza un **puntatore** all'indirizzo del contratto reale.
- Quando si vuole distribuire una **nuova versione** del contratto reale, l'indirizzo memorizzato nel contratto proxy viene aggiornato con quello della nuova versione del contratto.
- Questo consente di **sostituire** il contratto reale senza alterare l'interfaccia o l'indirizzo a cui gli utenti e il codice client fanno riferimento.

Migration

```
pragma solidity ^0.8.0;
import "../Ownable.sol";

contract Proxy is Ownable
{
    address internal _realContractAddress;

    function updateReference(address _newContractAddress) external onlyOwner
    {
        _realContractAddress = _newContractAddress;
    }

    fallback() external payable
    {
        (bool success, ) = _realContractAddress.delegatecall(msg.data);
        //cattura il valore della tupla restituita delegatecall (salvando solo il primo elemento)
        require(success, "Delegatecall failed");
    }

    receive() external payable {}
}
```

```
pragma solidity ^0.6.0;

contract RealService
{
    uint8 private constant _increment = 1;
    uint128 _value;

    function increment() public
    {
        _value += _increment;
    }

    function getValue() public view returns(uint128)
    {
        return _value;
    }
}
```

```

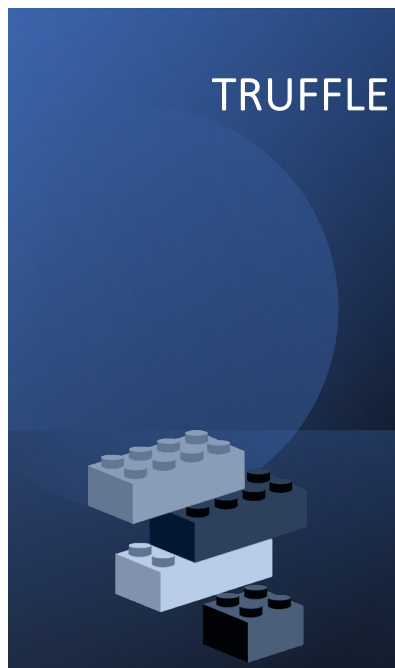
pragma solidity ^0.6.0;

contract UpdatedService
{
    uint8 private constant _increment = 10; //differenza con la prima versione
    uint128 _value;

    function increment() public
    {
        _value += _increment;
    }

    function getValue() public view returns(uint128)
    {
        return _value;
    }
}

```

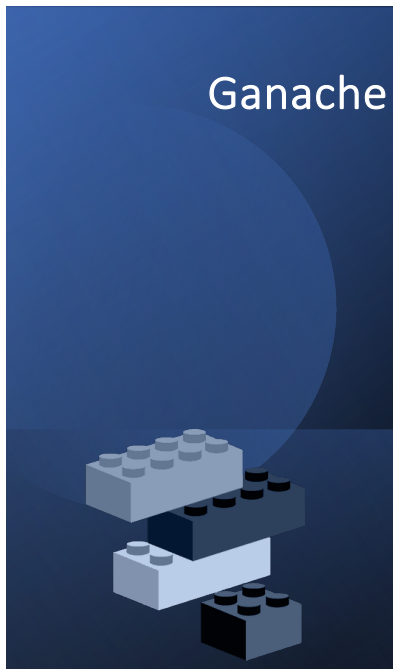


Truffle:

- **Compilazione, Migrazioni (deploy), Test (Web3.js)**
- `npm install -g truffle`
- `truffle init`: crea una nuova struttura di cartelle con tutti i file di configurazione necessari per iniziare a lavorare con Truffle.
- `truffle compile`: compila i contratti
- `truffle migrate`: esegue le migrazioni e distribuisce i contratti.
- `truffle test`: esegue i test scritti in javascript salvati nella cartella `test/`.

<https://trufflesuite.com/docs/>

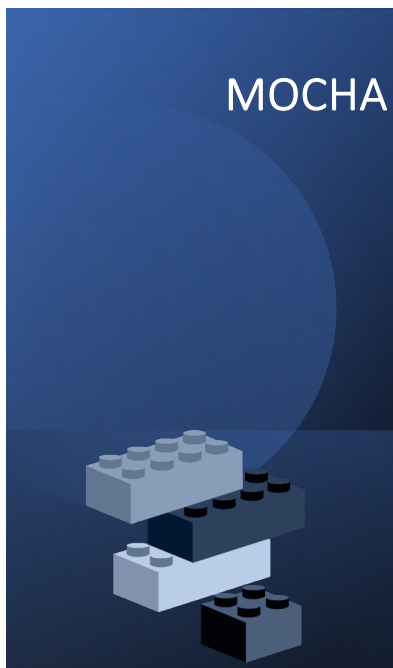




Ganache:

- **Emulatore:** Simula il funzionamento di un nodo Ethereum in un ambiente locale e controllato.
- **Interfaccia grafica:** permette di visualizzare le transazioni e gli stati dei contratti.

<https://trufflesuite.com/ganache/>

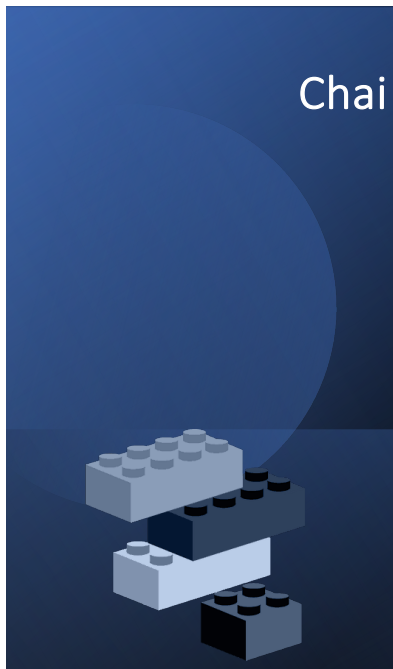


Mocha:

- npm install --save-dev mocha
- framework di test JavaScript
- rende i test asincroni semplici.
- mappa le eccezioni non catturate ai casi di test corretti.

<https://mochajs.org/>





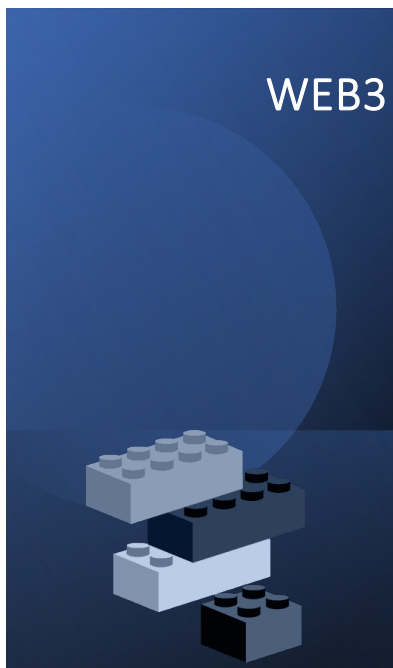
Chai:

- npm install --save-dev chai
- npm install --save-dev chai-as-promised
- framework di test JavaScript
- Behavior-Driven Development basati su catene.
 - Es: `expect(x).to.be.a('string');`
- Test-Driven Development più classica.
 - Es: `assert.equal(y, 123);`

<https://www.chaijs.com/>



Chai Assertion Library



Web3:

- npm install web3
- libreria JavaScript che interagisce con un nodo locale o remoto di Ethereum.
- Fornisce un'interfaccia per inviare transazioni, interagire con smart contract e accedere ai dati della blockchain.
- Utilizza una connessione HTTP, IPC o WebSocket.
 - IPC consente lo scambio di dati tra un client (ad esempio, un'applicazione che utilizza web3.js) e un server (come un nodo Ethereum).
 - WebSocket consente una comunicazione full-duplex continua e in tempo reale su una singola connessione TCP.

<https://web3js.readthedocs.io/en/v1.10.0/>



Testing

La funzione `contract` fornita da Truffle crea, per ogni contratto, suite di test separate mantenendo un ambiente “pulito” e isolato.

All'interno di queste suite, definiamo **specifici test case** utilizzando la funzione `it`.

Verifichiamo la corretta distribuzione dei contratti, utilizzando asserzioni per controllare che l'indirizzo di ciascun contratto sia valido e non nullo.

Nel caso del contratto Proxy, facendo affidamento sulle **asserzioni** `.should.be.fulfilled` e `.should.be.rejected`, esaminiamo anche la funzionalità di aggiornamento del puntatore al `RealService`, assicurandoci che solo l'account dell'owner possa aggiornarlo.

Usiamo `Web3.js` per configurare un'istanza collegata a Ganache (nodo di Ethereum locale), per interagire direttamente con i contratti tramite il Proxy. Creiamo un'istanza del contratto utilizzando l'ABI del `RealService` e l'indirizzo ottenuto dal contratto Proxy e verifichiamo che le funzioni `increment()` e `getValue()` reagiscano come previsto.

Dopo aver modificato il reference al `RealService` nel Proxy, rieseguiamo lo stesso test su `increment()` e `getValue()` verificando il diverso comportamento della nuova versione del Real (adesso Updated) Service.

Immutabilità

Emergency Stop:

- Meccanismo di sicurezza che permette di interrompere la maggior parte delle operazioni in caso di anomalie o scoperte di vulnerabilità, per prevenire potenziali (o ulteriori) danni bloccando le transazioni.
- Nonostante il blocco, funzioni essenziali come il prelievo di fondi possono rimanere attive per permettere agli utenti di recuperare i loro asset.

Emergency stop

```
import "./Ownable.sol";
contract Stoppable is Ownable
{
    uint private _locked;

    modifier onlyIfUnlocked()
    {
        require(_locked == 0, "The contract is in emergency stop state, try again later.");
        _;
    }

    function lock() public onlyOwner
    {
        _locked = 1;
    }
    function unlock() public onlyOwner
    {
        _locked = 0;
    }
}
```

GradualPonzi

```
pragma solidity ^0.8.0;

import "./Stoppable.sol";

contract GradualPonzi is Stoppable
{
    address[] public _investors;
    mapping (address => uint256) public _balances;
    uint256 public constant MINIMUM_INVESTMENT = 1e15;

    constructor ()
    {
        //l'owner è il primo investitore che beneficerà del sistema Ponzi, ricevendo una parte degli investimenti successivi.
        _investors.push(msg.sender);
    }
}
```

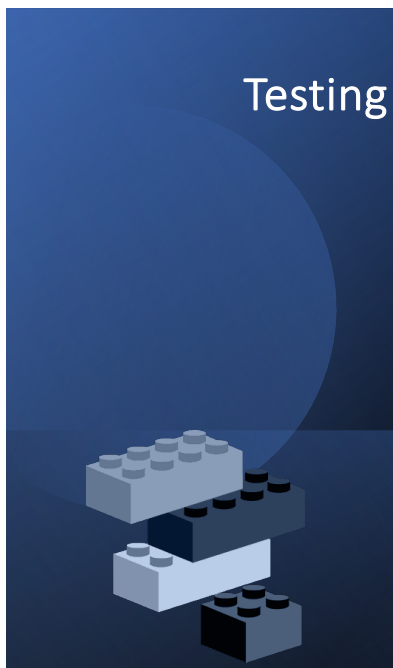
```

function invest() public payable onlyIfUnlocked
{
    require(msg.value >= MINIMUM_INVESTMENT);
    uint256 eachInvestorGets = msg.value / _investors.length;
    for (uint256 i=0; i < _investors.length; i++)
    {
        _balances[_investors[i]] += eachInvestorGets;
    }
    _investors.push(msg.sender);
}

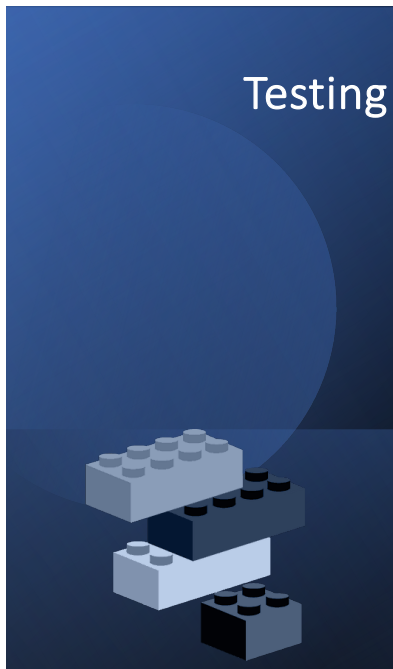
function withdraw() public
{
    uint256 payout = _balances[msg.sender];
    _balances[msg.sender] = 0;
    if(payout>0)
        payable(msg.sender).transfer(payout);
}

function debugGetBalance() public view returns (uint256)
{
    return _balances[msg.sender];
}

```

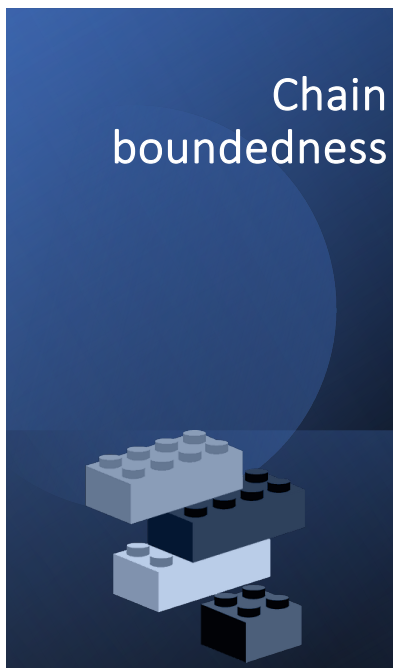


- Soglia: assicurarsi che investimenti minori della soglia minima non vengano accettati.
- Investimento: effettuare che dopo un investimento valido, l'indirizzo dell'investitore sia aggiunto a `_investors`, che il bilancio dell'investitore sia aggiornato correttamente e che gli eventi siano salvati correttamente.
- Distribuzione: assicurarsi che gli ETH investiti vengano distribuiti equamente tra gli investitori precedenti.
- Prelievo: testare che un investitore possa prelevare il proprio bilancio e che il bilancio venga azzerato dopo il prelievo.
- Blocco e Sblocco: testare che il proprietario possa bloccare e sbloccare il contratto e che gli altri utenti non possano farlo.
- Verifica del Lock: testare che la funzione investimento non sia eseguibile in seguito alla chiamata `lock()` e che il prelievo invece lo sia.



Testing

- **Web3.js**: Utilizzata per interagire con Ethereum blockchain, permettendo di invocare funzioni del contratto e ottenere informazioni sugli account.
- **BigNumber**: usa la libreria `bignumber.js` per gestire numeri di grandi dimensioni. È essenziale per confrontare correttamente i saldi e i trasferimenti di Ether.
 - `npm install bignumber.js`



Chain boundedness

Eventi:

- In Ethereum, gli eventi catturano informazioni e sono utilizzati come mezzo per comunicare con il **mondo esterno** (es. componenti di interfacce utente e server).
- Sono memorizzati sulla blockchain come parte della cronologia delle transazioni e possono essere recuperati anche dopo il completamento della transazione.
- Forniscono un modo **economico** per memorizzare e comunicare informazioni sulla blockchain. Rispetto all'aggiornamento di variabili di storage, gli eventi consumano meno gas.
- La dichiarazione di evento consiste nel nome, un elenco di parametri (se presenti) e i loro tipi di dati.
 - `event nome(tipo parametro1, tipo parametro2, ..., tipo parametroN);`
- I loro argomenti possono essere contrassegnati come `indexed`, consentendo di costruire indici per filtrare gli eventi e rendere più efficienti le ricerche.
- Per emettere un evento, si utilizza la parola chiave `emit`, seguita dal nome dell'evento e dai parametri necessari.
 - `emit nome(parametro1, parametro2, ...);`

Esercizio

- Aggiungere al contratto GradualPonzi.sol l'emissione di eventi quando un utente versa o preleva ETH.

Gradual Ponzi con eventi

```
.  
.   
.   
event Invested(address indexed investor, uint256 amount);  
event Withdrawn(address indexed investor, uint256 amount);  
.   
.   
.   
emit Invested(msg.sender, msg.value); //emissione dell'evento  
.   
.   
.   
emit Withdrawn(msg.sender, balance); //emissione dell'evento  
.   
.   
. 
```

Testing

- Testare che gli eventi salvati corrispondano ai dati attesi.
- **Lettura dei log degli eventi:** per verificare gli si devono leggere i log della transazione, iterando attraverso tutti gli eventi e cercando quello che interessa controllare.

```
transaction = await
GradualPonziInstance.invest(depositSufficient);

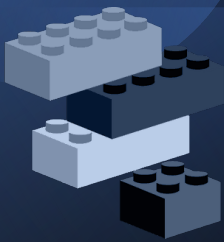
transaction.logs.forEach(log =>
{
    if (log.event === "Invested")
    {
        investor = log.args.investor;
        amount = log.args.amount;
    }
});
```

Costi

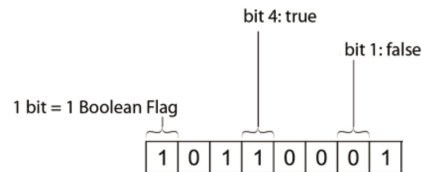
Best practices per ottimizzare il consumo di gas:

- **Impacchettamento:** dichiarare variabili di storage dello stesso tipo consecutivamente per consentire al compilatore di impacchettarle in meno slot.
- **int* vs uint256:** quando non è possibile impacchettare, utilizzare `uint256` anziché interi più piccoli per evitare inutili conversioni.
- **Limitare chiamate esterne:** utilizzare una singola funzione multiuso.
- **Mapping vs Array:** le tabelle hash sono più efficienti degli array classici.
- **Array a dimensione fissa:** non utilizzare array dinamici per evitare operazioni di gestione automatica.
- **Valore predefinito:** evitare inizializzazioni esplicite superflue in quanto Solidity imposta automaticamente le variabili a zero.
- **Short circuit:** ordinare le espressioni con operatori logici per ridurre la probabilità di valutare il secondo termine.
- **Stringhe brevi:** mantenere le stringhe costanti al di sotto di 32 byte.
- **Scambio in linea singola:** usare l'istruzione nativa di Solidity per scambiare valori di due variabili senza variabili ausiliarie. `(a, b) = (b, a);`
- **Valori hardcoded:** impostare valori noti invece di calcolarli per evitare calcoli inutili.

BooleanBox



- In Solidity (e in molti altri linguaggi), i Booleani sono memorizzati come interi senza segno di un byte (o più) perché i dati devono essere **indirizzabili**.
- BooleanBox è un metodo per ottenere otto valori booleani da ogni byte, compattando più informazioni piuttosto che utilizzare otto variabili diverse. Si possono ottenere 256 Booleani con una word della EVM tramite un array di 32 `uint8`.
- **Riduzione del costo del gas** per la memorizzazione di dati Booleani.
- Grazie alle operazioni bitwise, ogni bit può essere letto individualmente.



bit is the index of the specific Boolean Flag

BooleanBox



Funzionalità chiave:

- **set**: consente di impostare il valore di tutti gli 8 bit contemporaneamente.
- **get**: restituisce lo stato corrente di tutti gli 8 bit in una volta.
- **getFlag**: restituisce lo stato attuale di un bit specificato da un indice.
- **setFlagTrue**: permette di attivare un singolo bit indicato da un indice.
- **setFlagTrueMask**: usa una maschera per attivare più bit insieme.
- **setFlagFalse**: disattiva un singolo bit indicato da un indice.
- **setFlagFalseMask**: usa una maschera per disattivare più bit insieme.
- **toggleFlag**: inverte lo stato di un bit indicato da un indice.

Gli sviluppatori devono gestire con attenzione gli indici dei bit per evitare overflow e errori.

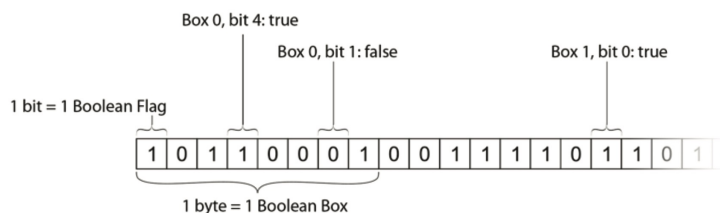
Testing

- **Impostazione di un bit specifico:** utilizzare la funzione `setFlagTrue` e `setFlagFalse` per impostare un bit specifico a 1 o a 0. Successivamente, verificare che solo il bit desiderato sia stato modificato, mantenendo inalterati gli altri.
- **Impostazione di bit con maschere:** usare le funzioni `setFlagTrueMask` e `setFlagFalseMask` per impostare più bit contemporaneamente usando una maschera binaria. Verificare che dopo l'applicazione della maschera, i bit del `BooleanBox`, che corrispondono agli 1 della maschera, siano stati modificati.
- **Toggle di un Bit specifico:** verificare la capacità di invertire lo stato di un bit specifico, in entrambe le direzioni, utilizzando la funzione `toggleFlag`.
- **Lettura del byte:** tramite la funzione `get`, verificare il valore del `BooleanBox`.
- **Lettura dello di un Bit specifico:** tramite `getFlag`, verificare che la lettura dello stato di un bit specifico rifletta il valore attuale di quel bit.

BooleanBoxes

Funzionalità chiave:

- È come il `BooleanBox` ma usa un array di 32 byte (256 bit) per occupare un'intera word della EVM.
- `uint8[32] myBooleanBoxArray; //256 bits`
- Le funzioni sono uguali ma si deve specificare quale "box" (byte) usare, tramite un indice aggiuntivo



n is the index of the Boolean Box array
bit is the index of the specific Boolean Flag

Esercizio

- Creare dei test per BooleanBoxes.sol.

Testnet



Sepolia



Rete di Test Ethereum:

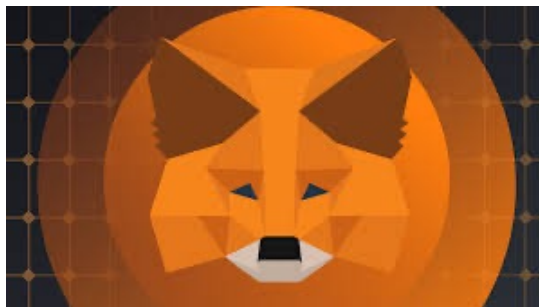
- Sepolia è una delle reti di test di Ethereum, utilizzata per testare smart contract prima del loro lancio sulla rete principale.
- Sostituisce reti di test precedenti come Ropsten.
- Non utilizza Ether reale, ma un Ether specifico della rete di test.
- Supporta Proof of Stake, aiutando a simulare e testare l'ambiente post-Merge.

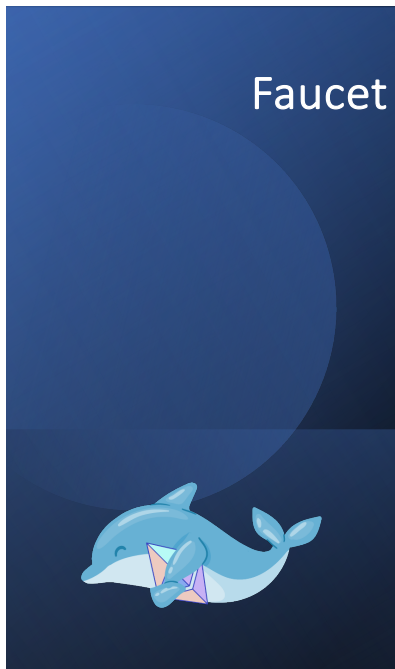
Metamask



Metamask:

- MetaMask è un wallet software e un gateway per interagire con applicazioni decentralizzate sulla blockchain di Ethereum.
- Permette di interagire con diverse blockchain.
- Si usa attraverso il browser.
- <https://metamask.io>



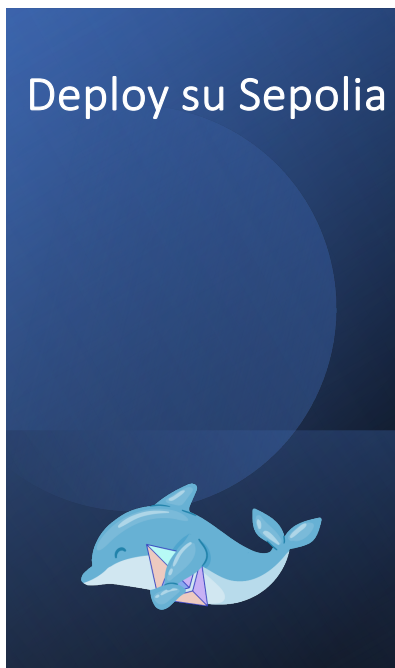


Faucet

Faucet di Ethereum su Sepolia:

- I faucet sono servizi che distribuiscono gratuitamente Ether di test agli sviluppatori per permettere l'esecuzione di test sulle reti.
- Erogano una quantità limitata di Ether.
- Spesso sono implementati da siti che offrono servizi legati allo sviluppo di smart contract.

- <https://cloud.google.com/application/web3/faucet/ethereum/sepolia>
- <https://www.alchemy.com/faucets/ethereum-sepolia>



Deploy su Sepolia

Deploy da Remix:

- Assicurati che MetaMask sia impostato su Sepolia e che il wallet sia connesso a Remix.
- Nella scheda Deploy & run transactions, selezionare **Injected Web3** come ambiente per usare MetaMask.
- Scegliere il contratto e cliccare su Deploy.
- Confermare la transazione su MetaMask.